Release — Software Design

Document ID	SDS-REL-001
Revision	dfa6eda
Date	2026-11-10
Status	Controlled in git

Contents

1 Purpose 3
2 Design Outputs 3
2.1 Order 4
3 Dependencies 4

Id: `SDS-REL-001` · Context: `release`

1 Purpose

`release` decides whether a release may go ahead and keeps the evidence it went ahead on: the release gate over the record's verified status, its risks and its user needs; the verification data the gate and its reviewers read; and the reusable gates that run all of it in another repository's pipeline. Its language: a requirement is *verified* (a passing run claims it, and none that claims it failed); the *release gate* blocks a release; *release-grade evidence* is what a release needs; the *traceability matrix* is generated, never edited; the *evidence bundle* is what a release keeps. It reads the specification, test evidence and risk; none of them reads it.

2 Design Outputs

Two of the components run in the `rdm` command, two in the reusable CI that other repositories' pipelines call.

Component	Responsibility	Meets
Release gate	The release decision over verified status, risk and validation	item-04-001; realises item-01-004, item-02-002, item-02-003
Verification data	Every requirement reconciled against the executed results: the data the traceability matrix and the evidence bundle are built from	item-04-002, item-04-003
Reusable workflow	Runs a repository's acceptance tests, then the gates, with RDM installed from the revision the caller pinned	item-04-004
Gates action	The gate steps, written once: verification data, release gate, graph validation and the evidence bundle	item-04-004, item-04-003

Release gate. It blocks a release when no spec is found, or the spec cannot be read whole (a scenario with no requirement id tag, an id tagging more than one scenario, a file that does not parse); when the spec holds no requirement; when a requirement failed or is untested in the given results; when a result file in them cannot be read (it could hold a failed run); when the risk register's release rules report a blocking finding; and when a user need is addressed by no requirement. It warns, never blocks, on a user need with no approved validation record, and on a

claimed tag that looks like a requirement id but names none. A results directory that does not exist is refused.

Verification data. For each requirement: its scenario, rule, source, context, status, run counts, tests and their outputs, grouped under the user needs it addresses (each with its text), with a summary, the tags that match no requirement, and the result files that cannot be read, which make `verify` exit non-zero. It is generated, never edited.

Reusable workflow and Gates action. The caller pins the workflow and names the same revision again as an input, because a reusable workflow cannot see the revision it was called at. The workflow checks out the caller's repository with its history and RDM at that revision, installs RDM and the test tools from that checkout (never from a package index), runs the caller's acceptance tests, then the Gates action, and uploads the Allure report and the verification record. The Gates action installs RDM from its own revision unless the caller already has; each gate can be switched off for a record that is not yet ready for it; the verification data and the evidence bundle run only when there are results, and the bundle is uploaded for retention. Inputs reach the steps' scripts as environment variables, never spliced into the script text, so an input cannot inject shell.

Relationships that matter, in the direction of the arrow:

- The release gate reads the spec and the validation records (`specification`), reconciles results with the Allure reader (`test_evidence`) and reports the risk findings of the risk register (`risk`).
- Verification data reconciles results with the Allure reader and reads the spec.
- The Reusable workflow runs the acceptance test run and then the gates with the Gates action, which writes the verification data, runs the release gate, validates the graph with SHACL validation (`graph`) and writes the evidence bundle with the Evidence bundle (`publishing`).

The release gate does not decide whether the runs are release-grade evidence (the commit and worktree each run tested): the verification report and, as a warning, the knowledge graph's gate shapes show it.

2.1 Order

The order of a pipeline run matters: the acceptance tests write the Allure results; the verification data is written; the release gate decides; graph validation runs; the evidence bundle is written last, from the same results, and uploaded.

3 Dependencies

Layer 4 of the dependency rule. Depends on `test_evidence` (the Allure reader), `specification` (the spec, the validation records) and `risk` (the risk register), all below it, and on the shared kernel. Depended on by `publishing` (the verification report and the evidence bundle build on the verification data) and by the composition root. The Gates action also runs `graph`'s SHACL validation and `publishing`'s evidence bundle as commands; that is a run, not an import, so the rule is not broken.